

Практичні приклади до уроку 9.3 - Швидкість API, HTTP-кешування та підготовка до highload

Загальна ідея практики:

Доповнити оптимізацію БД/кеша (9.2) **HTTP-рівнем**: Cache-Control, контроль об'єму JSON-відповідей, rate limiting та винесення важких операцій у jobs.

Приклади орієнтовані на фінансовий домен (payments/accounts/reports), але легко адаптуються під інші.

1. Блок 3: HTTP-кеш → Слайд 9а

• Загальна ідея:

Показати, які endpoint'и фінансового API логічно кешувати на рівні HTTP, і як підібрати заголовки Cache-Control для різних типів даних.

1.1. Пример: список платежей счёта (GET /api/v1/accounts/{id}/payments)

Это **персонализированный, часто меняющийся** список.

- Цель: слегка разгрузить повторные запросы UI (например, при навигации назад/вперёд), но не держать старые данные долго.
- Возможный заголовок:

```
Cache-Control: private, max-age=15
```

- Смысл:
 - **private**: кеш только на клиенте (браузер/мобильное приложение), не в общих прокси;
 - **max-age=15**: в пределах 15 секунд повторный запрос может отдать локальную копию.

1.2. Пример: справочник валют (GET /api/v1/currencies)

Это **публичный, редко изменяемый** ресурс.

- Возможный заголовок:

```
Cache-Control: public, max-age=3600
```

- Смысл:
 - **public**: можно кешировать на любом уровне (браузер, прокси, CDN);
 - **max-age=3600**: обновление не чаще одного раза в час.

1.3. Пример установки заголовков в Laravel

```

<?php

declare(strict_types=1);

namespace App\Http\Controllers\Api\V1;

use App\Http\Controllers\Controller;
use App\Http\Resources\Api\V1\CurrencyResource;
use App\Models\Currency;
use Illuminate\Http\JsonResponse;

final class CurrencyController extends Controller
{
    public function index(): JsonResponse
    {
        $currencies = Currency::query()->orderBy('code')->get();

        $response = response()-
>json(CurrencyResource::collection($currencies));

        $response->header('Cache-Control', 'public, max-age=3600');

        return $response;
    }
}

```

Что проговорить:

- HTTP-кеш не заменяет серверный Redis-кеш, а дополняет его;
- для критичных real-time данных (баланс) - скорее **no-store** или очень короткий **private** с чёткой логикой обновления.

2. Блок 4: Об'єм відповіді → Слайд 11а

• Загальна ідея:

На примере списка платежей показать, как **урезать** JSON-ответ до нужного минимума через Resources/DTO, уменьшая объём данных и ускоряя API.

2.1. Пример "жирного" ответа (идея)

```

{
  "data": [
    {
      "id": 123,
      "account_id": 10,
      "user_id": 5,
      "amount": "250.00",
      "currency": "USD",
      "description": "Оплата рахунку №1001",
      "status": "processed",
    }
  ]
}

```

```

        "gateway_payment_id": "gw_abc",
        "gateway_raw_response": "{...}",
        "internal_note": "debug...",
        "created_at": "2026-02-12T10:00:00Z",
        "updated_at": "2026-02-12T10:00:05Z"
    }
]
}

```

- Проблемы:
 - `gateway_raw_response`, `internal_note` и `updated_at` не нужны клиентскому UI;
 - лишние поля увеличивают размер и могут раскрывать внутренние детали.

2.2. Пример “чистого” ресурса для списка

```

<?php

declare(strict_types=1);

namespace App\Http\Resources\Api\V1;

use Illuminate\Http\Request;
use Illuminate\Http\Resources\Json\JsonResource;

/**
 * @property int $id
 * @property int $account_id
 * @property string $amount
 * @property string $currency
 * @property string|null $description
 * @property string $status
 * @property \Carbon\CarbonInterface|null $created_at
 */
final class PaymentListItemResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->id,
            'account_id' => $this->account_id,
            'amount' => $this->amount,
            'currency' => $this->currency,
            'description' => $this->description,
            'status' => $this->status,
            'created_at' => $this->created_at?->toIso8601String(),
        ];
    }
}

```

- посмотреть на свои ответы со списками (платежи, счета, отчёты);
- выписать поля, которые действительно нужны клиенту;
- предложить Resource/DTO, который это отражает и не “утекает” внутренними данными.

3. Блок 5: Rate limiting → Слайд 13а

- **Загальна ідея:**

Показать, как в Laravel настроить **разные лимиты** для обычных API-методов и тяжёлых операций (экспорт/отчёты).

3.1. Стандартный лимит для большинства API

```
<?php
```

```
use App\Http\Controllers\Api\V1\AccountPaymentsController;
use App\Http\Controllers\Api\V1\PaymentsController;
use Illuminate\Support\Facades\Route;

Route::middleware(['auth:sanctum', 'throttle:60,1'])
    ->prefix('api/v1')
    ->group(static function (): void {
        Route::get('accounts/{account}/payments',
[AccountPaymentsController::class, 'index']);
        Route::post('payments', [PaymentsController::class, 'store']);
    });
```

- **throttle:60,1** → 60 запросов в минуту на пользователя/токен.

3.2. Более жёсткий лимит для тяжёлых операций

```
<?php
```

```
use App\Http\Controllers\Api\V1\ReportsController;

Route::middleware(['auth:sanctum', 'throttle:5,1'])
    ->prefix('api/v1')
    ->group(static function (): void {
        Route::post('reports/account-statement',
[ReportsController::class, 'generate']);
    });
```

- **throttle:5,1** → не более 5 запросов на генерацию отчёта в минуту на пользователя.

Що пояснити:

- лимиты можно настраивать по ролям/типам endpoint'ов;
- front должен корректно обрабатывать 429 (например, показывать сообщение и предлагать повтор позже).

4. Блок 6: Jobs для тяжёлых операций → Слайд 15а

- **Загальна ідея:**

Перевести тяжёлую операцию (экспорт выписки) из синхронной модели “ждём PDF/CSV” в асинхронную “задача принята, результат будет позже”.

4.1. Пример: job для экспорта выписки (Laravel, скелет)

```
<?php

declare(strict_types=1);

namespace App\Jobs;

use App\Models\Account;
use Illuminate\Bus\Queueable;
use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Foundation\Bus\Dispatchable;
use Illuminate\Queue\InteractsWithQueue;
use Illuminate\Queue\SerializesModels;

final class ExportAccountStatementJob implements ShouldQueue
{
    use Dispatchable;
    use InteractsWithQueue;
    use Queueable;
    use SerializesModels;

    public function __construct(
        private readonly int $accountId,
        private readonly string $periodFrom,
        private readonly string $periodTo,
        private readonly int $requestedByUserId
    ) {
    }

    public function handle(): void
    {
        /** @var Account $account */
        $account = Account::query()->findOrFail($this->accountId);

        // TODO: собрать операции, сгенерировать CSV/PDF, сохранить в
хранилище,
        // создать запись с путём к файлу, отправить уведомление
пользователю.
    }
}
```

4.2. Контроллер, который ставит job и быстро отвечает

```

<?php

declare(strict_types=1);

namespace App\Http\Controllers\Api\V1;

use App\Http\Controllers\Controller;
use App\Jobs\ExportAccountStatementJob;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;

final class ReportsController extends Controller
{
    public function generateAccountStatement(Request $request):
    JsonResponse
    {
        $validated = $request->validate([
            'account_id' => ['required', 'integer'],
            'from' => ['required', 'date'],
            'to' => ['required', 'date', 'after_or_equal:from'],
        ]);

        // Создаём запись о задаче/репорт-запросе (упростим до id job'a)
        ExportAccountStatementJob::dispatch(
            (int) $validated['account_id'],
            (string) $validated['from'],
            (string) $validated['to'],
            (int) $request->user()->id
        );

        return response()->json([
            'status' => 'accepted',
            'message' => 'Report generation started.',
        ], 202);
    }
}

```

Що проговорити:

- API відповідає быстро (202), не блокується на генерації отчёта;
- воркер оброблює важку задачу в фоні;
- дальнійший шаг - endpoint/status или нотифікація о готовності результату.

5. Блок 7/8: План дій для власного API

• Загальна ідея:

Зафіксувати, в якому стані зараз ваш API і які **2–3 шага** дадуть найбільший ефект по швидкості/стабільності.

5.1. Міні-чек-лист для свого проекту

Для каждого из ключевых endpoint'ов (например, платежи, счета, отчёты) ответьте:

- Есть ли:
 - пагинация?
 - серверный кеш (Redis) там, где он безопасен?
 - HTTP-кеш (для публичных/редко меняющихся данных)?
 - rate limiting?
 - вынесение тяжёлых операций в jobs?
- Что уже сделано из:
 - индексы, отсутствие N+1, измерения с профайлерами?
 - логи и Sentry для отслеживания проблем под нагрузкой?

Що зробити на практиці:

- выбрать 2–3 пункта, которые вы можете реально внедрить в ближайшие 1–2 недели;
- связать их с измерениями: какой эффект вы ожидаете и как его проверите (время ответа, ошибки, нагрузка).

6. Примітки щодо середовища / docker-compose

- Для HTTP-кеша и rate limiting **не требуются** изменения в **docker-compose** - это уровень приложения и, возможно, внешнего прокси/CDN.
- Для jobs необходимо, чтобы в docker-стеке был настроен воркер очереди (как в Модуле 5); для профилирования под нагрузкой - стабильное окружение и мониторинг (Модуль 8).

Головна ідея практичної частини уроку 9.3:

Завершая программу, вы увязываете **архитектуру, тесты, логи, кеш, БД и HTTP-уровень** в единую картину production-ready API. Конкретные small-steps (HTTP-кеш для справочников, ограничение ответов, лимиты, jobs для тяжёлых задач) дают реальный, измеримый выигрыш и подготавливают ваш бекенд к росту нагрузки без паники и «переписывания всего с нуля».